

Winter Milestone Compliance Report 3

GK02: Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming

Project Manager:

Davis Cheung - 501014636

Team Members:

Victor Do - 501137174

Cameron Tuffner-Lyons - 501029609

JJ Marr - 500879103

Faculty Supervisor

Dr. Gul N. Khan

March 13th, 2025

Abstract:

This milestone report details significant advancements in our fluid simulation capstone project. Key achievements include substantial performance improvements through CPU-GPU task partitioning, averaging a 115% increase and reaching up to 1442% with optimized fixed-time steps, and WebGPU compatibility for enhanced accessibility. The simulation now incorporates an entropy system, asynchronous readbacks, and a sensor manager for efficient thermal data handling. New features include dynamic obstacle spawning with context menus, a particle cap, and diverse fluid types like fire, oil, crude oil, smoke, and cold fire, each with distinct physical and visual properties. User interface enhancements include improved menus, sliders for brush control, and visual feedback. Game content progress includes level 2 improvements, preliminary implementation of level 3, and planning for levels 4 and 5. Challenges addressed include WebGPU porting issues, viscosity-related clumping, high-pressure instability, and rigorous GPU memory management.

Tasks Outlined & Group Member Contributions:

Student Name:	Github and Functional Tasks Completed	Report Sections
Student A: (VD)	- add blur to starting scene text bg - FREE GPU MEMORY CORRECTLY, IF YOU SPAM RELOAD IT ACCUMULATES - bug fix particle temp not diffusing - bug fix fire not disabling - Remove the separate thermal box collision in favor of just reusing the regular box collisions - Update the CPU-GPU's compute shader to have the entropy changes - Update the CPU kernels to use the fluid ID functionality - adjusted Fire particles - Fix viscosity clumping issue - FREE MEMORY WHEN UPDATING FLUIDS EACH FRAME PROPERLY - bug fix the the fluid index not working in the visual shader and spawning kernels - Carnival water gun fill up target level - Update project to Unity 6 and BUGFIX - Create oil fuel type (flammable) - Create crude oil fuel type that burns with smoke - Create smoke fuel type and add to crude oil simulation - Create separate categorical submenus on the right for liquids, gases, powders and obstacles which activate on hover of mouse or click - improved stability of lava and honey and other high pressure force fluids on low performance systems - updated ui to scroll automatically depending on mouse position for long lists of fluids - added big red arrows to level 2 - added animated crosshair to level 2 - added slider for adjusting the brush interaction strength - added slider for adjusting brush size - added shortcut to adjust brush interaction strength - added UI interface around mouse when adjusting interaction strength - added an eraser brush for mobile platforms who may not have right click - added ability to spawn and delete obstacles in real time - fixed bug where disabled particle positions were not being randomized - added blur shader to some ui elements - added cold flame particle type for cooling down fluids - added the ui panels for spawning in different types of obstacle prefabs - improve the color gradient animation on thermal boxes and made them turn blue when below room temperature - added context menus on right click for adjusting the properties of obstacles in real time - added setting to show or hide the mouse circle - adjustments to sandbox scene starting properties - added custom context menus for source and thermal objects - added platform specific warning messages in the settings menu - added separate source object spawn rate from mouse spawn rate and tied mouse spawn rate to the interaction strength property - Fixed frame counter issue which lead to the first 10 frames of simulation not being skipped, which lead to highly inconsistent frame times at the start, which lead to the simulation breaking entirely occasionally - fixed obstacle resizing bug - added FPS counter display in top left corner	

Student B: (JM)	- Unity 6 and WebGPU support - Fix WebGPU load scene asynchronously - Setup Website (thefluidtoy.com)	
Student C: (CTL)	- Performance benchmarking of fluid simulation to determine performance uplift/loss in different scenarios. - Create test scene with 4 varying size big circle colliders and then make like 500 large green fluid particles to showcase cpu and gpu colliders more clearly, make the green fluid HUGE in size so you can clearly see their circular shape - Adjust entropy loss on per particle basis based on its diffusivity, this way more conductive particles lose temp faster than less thermally conductive ones - switch from waiting to read particle buffer for fluid detector to using async read back call for better web support - Carnival water gun fill up target level - Final CPU-GPU Implementation on UE6 CPU-GPU Integration - Created purely GPU physics script for performance comparisons - working on level 3	
Student D: (DC)	- Reduced SpatialIndices from uint3 to SpatialKeys which is uint1 - Added Temperature-changing game object (heater/cooler) - Added Entropy system supporting scene/fluid/static temperature targets - Add Fire particles which increase temperature of surrounding particles - Fixed improper particle type indexing in CPU-GPU scripts after type refactor - Altered particle spawner to add ability to place different fluids in the map at different areas on level start - Added a max particle count + scene fill with disabled particles - Added temperature sensors - Added a sensor manager for performing a single async readback across all particle sensors in simulation, refactored particle sensors	

Table 1: This is a breakdown of the specific tasks each member worked on.

February 11, 2025 – March 11, 2025

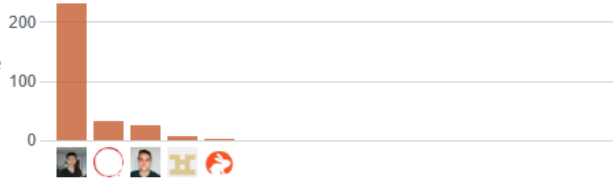
Period: 1 month ▾

Overview



6 Merged pull requests	2 Open pull requests	50 Closed issues	17 New issues
---------------------------	-------------------------	---------------------	------------------

Excluding merges, **5 authors** have pushed **268 commits** to main and **295 commits** to all branches. On main, **300 files** have changed and there have been **105,786 additions** and **15,936 deletions**.



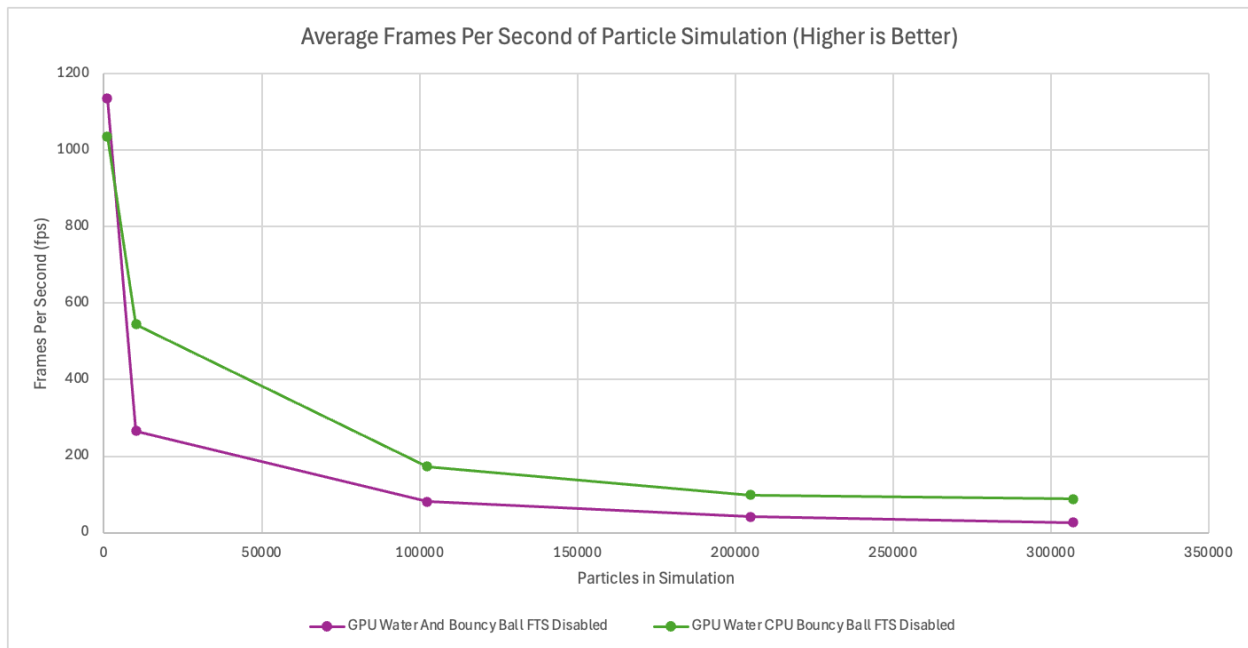
6 Pull requests merged by 4 people

Figure 1: This is a breakdown of closed issues and pull requests over the past month.

Performance Benchmarks:

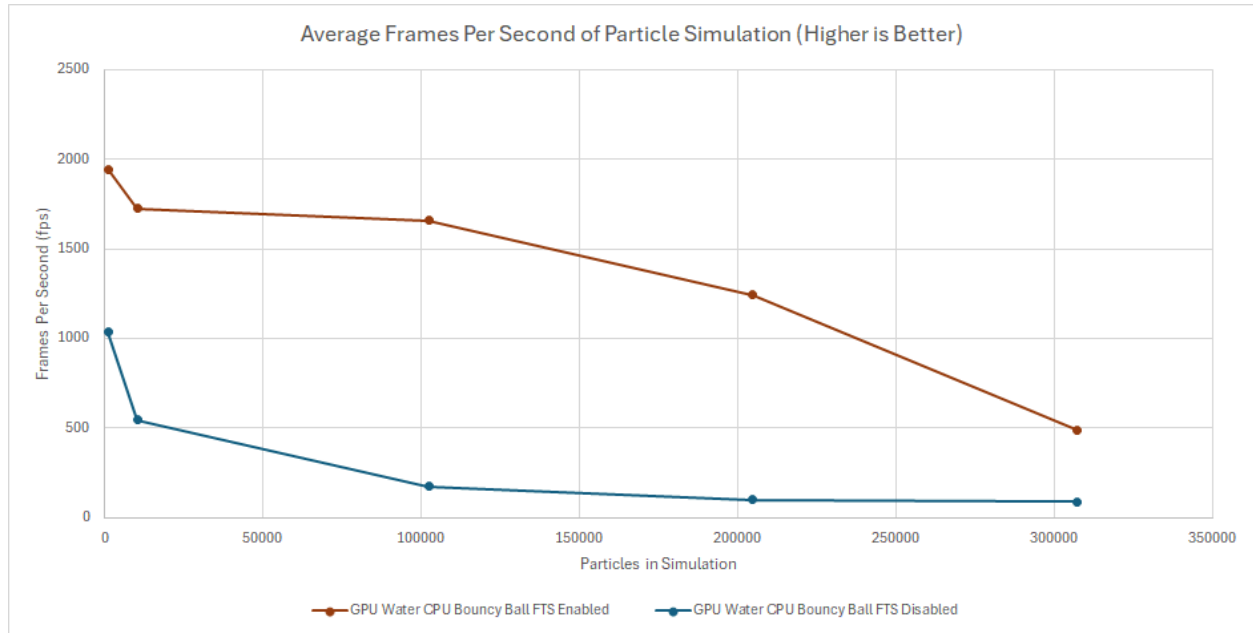
We have collected extensive data on the performance of the capstone project with respect to the total number of particles being simulated, the types of particles being simulated, and a simulation feature called *Fixed Time Step* (FTS). Fixed time step is a feature which forces simulation updates to occur at a fixed rate, rather than updating with every frame. This feature increases simulation quality and potentially performance depending on the scene being rendered. The full performance benchmarking dataset can be found in the appendix of this report. The benchmarking data shows the performance of the simulation in several different scenarios, however this report will focus on the most important results. The first result this report shows is that performance improvements can be obtained by placing particles of a significantly different size onto the CPU for physics calculations.

Graph 1: Average FPS of the Particle Simulation with GPU Water and Bouncy Ball Particles and GPU Water with CPU Bouncy Balls



When we move the “Bouncy Ball” particles, which are very large particles that behave elastically, onto the CPU we see a consistent performance improvement as the total number of particles increases. This is due to the broad phase efficiency reductions that are caused by placing particles of varying size on the GPU. A GPU performing uniform spatial subdivision must define the subdivision cell size equal to the maximum size of object which it must detect collisions for. However, this increase in subdivision cell size significantly reduces the broad phase collision detection efficiency for smaller particles. The performance improvement by moving the collision detection for larger particles to the CPU is shown on Graph 1 and averages to 115% improvement on average. This improvement is limited primarily by the CPU's collision detection performance, and the overhead in transferring the position information from the CPU to the GPU.

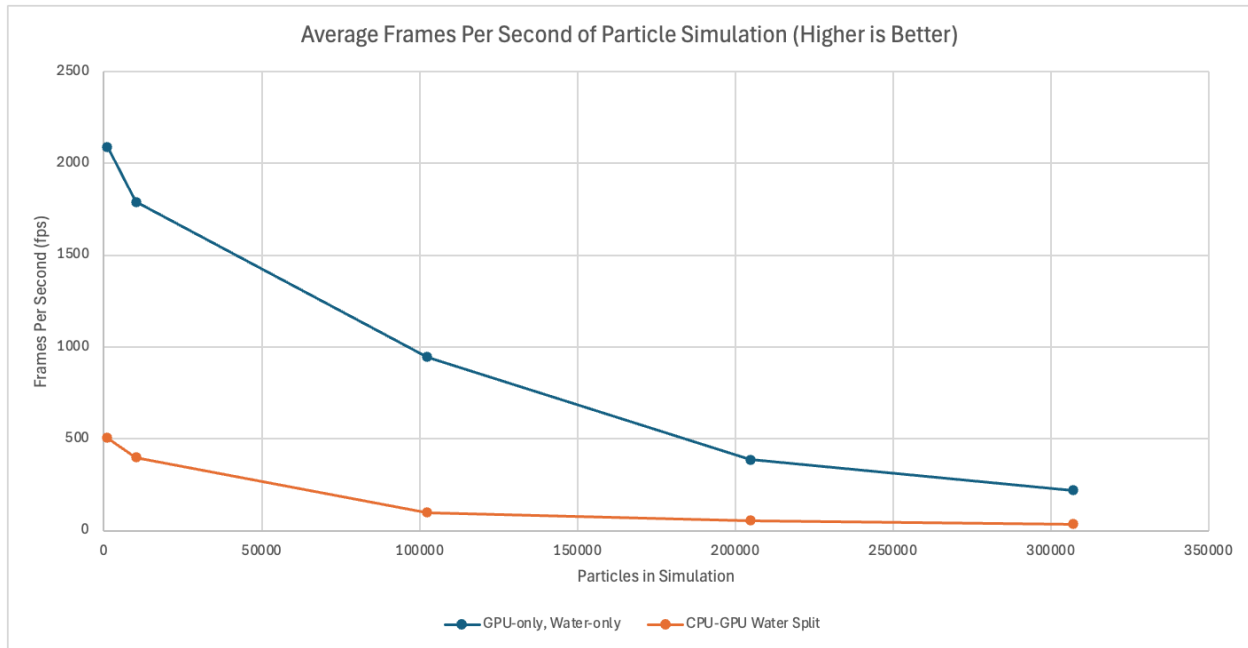
Graph 2: Average FPS of the Particle Simulation for GPU Water Particles and CPU Bouncy Ball Particles With FTS Enabled and Disabled



Graph 2 demonstrates the benefit of fixed time steps with respect to CPU-GPU computation. The implementation of fixed time steps is seen to improve the performance of the simulation. The average performance increase of fixed time step was 556% giving a total performance uplift compared to baseline (GPU Water and Bouncy Balls with FTS Disabled) of 1442%, with diminishing returns as the particle count was increased. The “GPU Water, CPU Bouncy with FTS enabled” performance represents the best performance of the simulation with two particles of significantly different size.

It is worth noting that the performance of the “GPU water only” simulation had the best overall performance, allowing the GPU to use an optimal spatial resolution and evenly divide the workload with high coherency during execution.

Graph 3: Performance Reduction of a CPU-GPU Particle Split With Only 1 Fluid Type



The performance reduction between the “GPU-only” and “CPU-GPU” water simulations was the expected result from these tests. Splitting the fluid simulation with only 1 fluid type across the CPU and GPU results in a reduction in performance due to the data transfers required to implement such a split. Contrasted with Graph 2 which has all of a type of fluid on a single processing element, and only sharing positions from CPU to GPU.

Game Features and Enhancements:

Particle Entropy

Alongside the addition of “fire” particles, a required feature was particle entropy. Particle entropy was achieved through setting a target entropy temperature, a property tied to each fluid which may vary by scene, be disabled, or otherwise be fixed to a specific value. This value is also tied with the particle’s thermal diffusivity, and determines the temperature particles will attempt to approach over each simulation step regardless of interactions or lack thereof with other particles; simulating thermal entropy.

Asynchronous Particle Readbacks

Asynchronous particle readbacks were implemented to allow for webGPU compatibility as well as improve performance. In Unity, asynchronous readback requests are implemented similarly to interrupt requests [1]. When calling an asynchronous readback of data from the GPU, both the GPU buffer and handler function can be specified at request. Once the request is completed, the handler function is called which accesses the data and does work [1]. Further

improvements on asynchronous readback handling can be found in the next section; Thermal Sensor and Sensor Manager.

Thermal Sensors and Sensor Manager

Particle sensors were partially refactored with an abstract class in order to support sensor management and make sensors easier to create; while thermal sensors were added to detect particle temperature. A sensor manager was written to reduce the number of asynchronous requests to GPU to a maximum of once per check period, instead of once per check period per fluid sensor. This can drastically improve performance when multiple sensors are used in the scene, as each request requires the transfer of all particle data from the GPU to the CPU, which scales poorly with many sensors.

Multiple Fluid Spawning and Particle Cap

The base fluid spawner was refactored and the spawning script lifted from the main simulation script in order to make it both simulation-agnostic (functions on both GPU-only and CPU-GPU scripts without refactor) as well as more scalable for multiple fluids. This enables different particle types to be loaded into levels much more easily using a simple rectangular spawner.

Additionally, a formal particle cap was imposed (previously dependent on the spawner script), which enforces a fixed number of particles in simulation on scene load. Warnings are thrown if spawners try to spawn more than the allotted amount, while invisible “disabled” particles are distributed evenly across the scene to prevent density hotspots if the total is below the cap.

Context Menus

In previous versions of the application we needed access to the Unity editor in order to change the properties of obstacles and so it wasn't possible for players to access in real time. Now, we have added in-game context menus which can be accessed by right clicking each obstacle, after changing one of the properties it triggers a callback which rebuilds the structured buffers with the updated properties. This is discussed more in the section about GPU memory management.

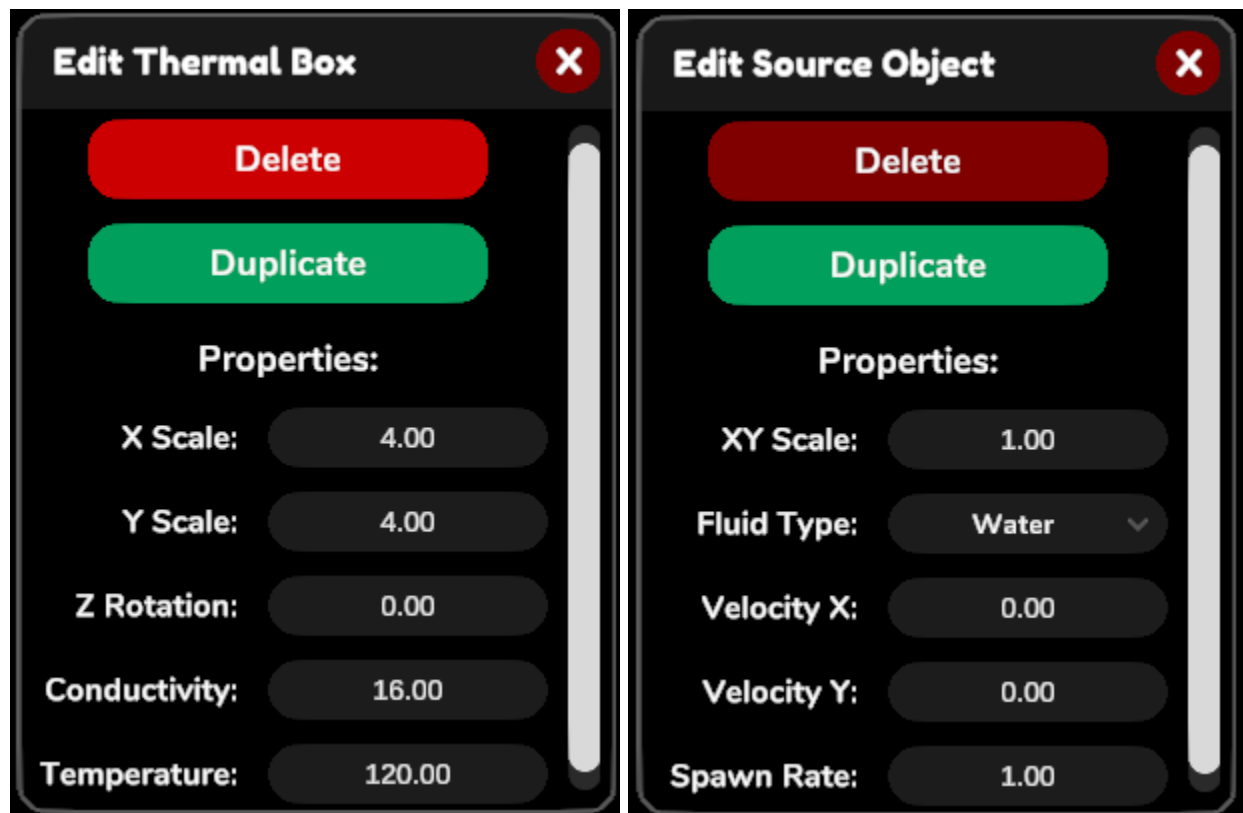


Figure 2: Context menus which can alter obstacle properties in realtime

Game Content:

New Fluid Types

1. Fire

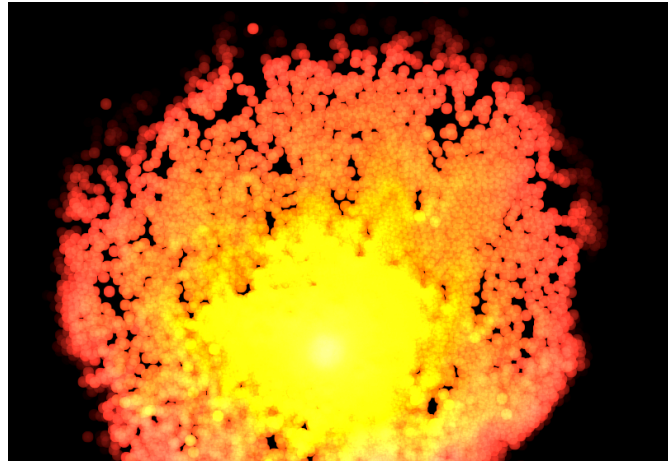


Figure 3: Fire particles in the sandbox

Physical Properties	Visual Properties
• Very light with low density and gas-like properties • Incredibly high base temperature • Rapidly “burns out” by decreasing in temperature over time due to entropy, before disappearing	• Yellow-white to red colour, depending on temperature • Fades to black as it cools to prevent sudden disappearing effect • Glows

2. Oil

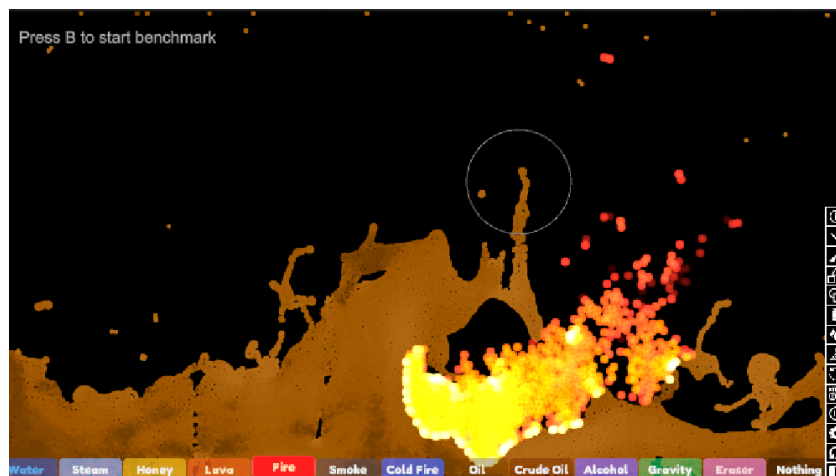


Figure 4: Ignited oil in the sandbox

Physical Properties	Visual Properties
• Viscous fluid • Less Dense than water and will settle on top • Upper-bound “state change” ignites the fluid, turning particles in fire particles, which ignite surrounding particles in a chain reaction	• Light brown • Uses velocity shader to change tone when moved quickly

3. Crude Oil

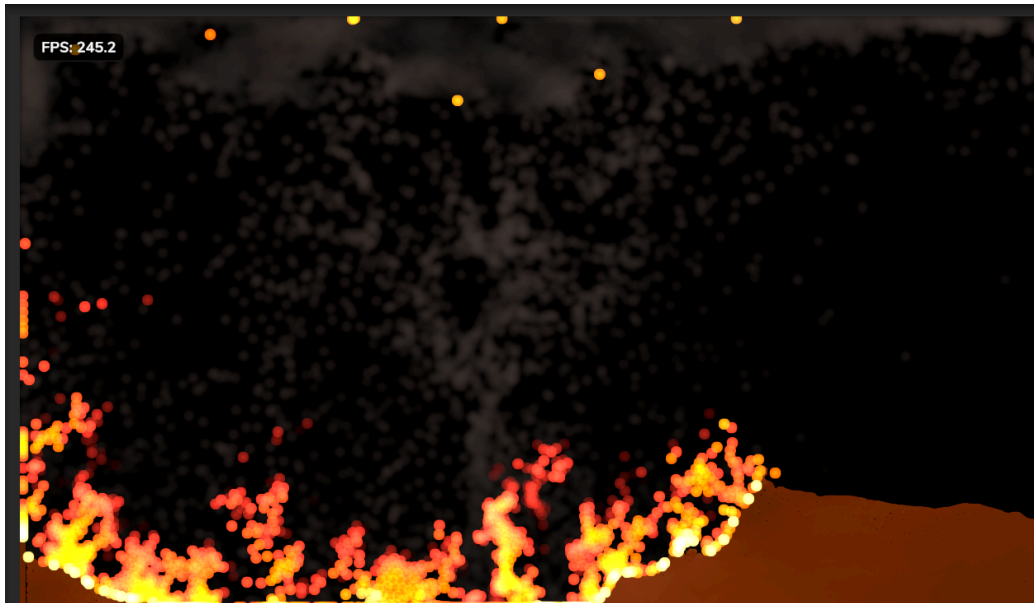


Figure 5: Ignited crude oil in the sandbox

Physical Properties	Visual Properties
• Very viscous fluid • Particles are flammable, if ignited they will transition into fire particles in a chain reaction before transforming once again to smoke particles.	• Dark, reddish-brown • Uses velocity shader to change tone when moved quickly

4. Smoke

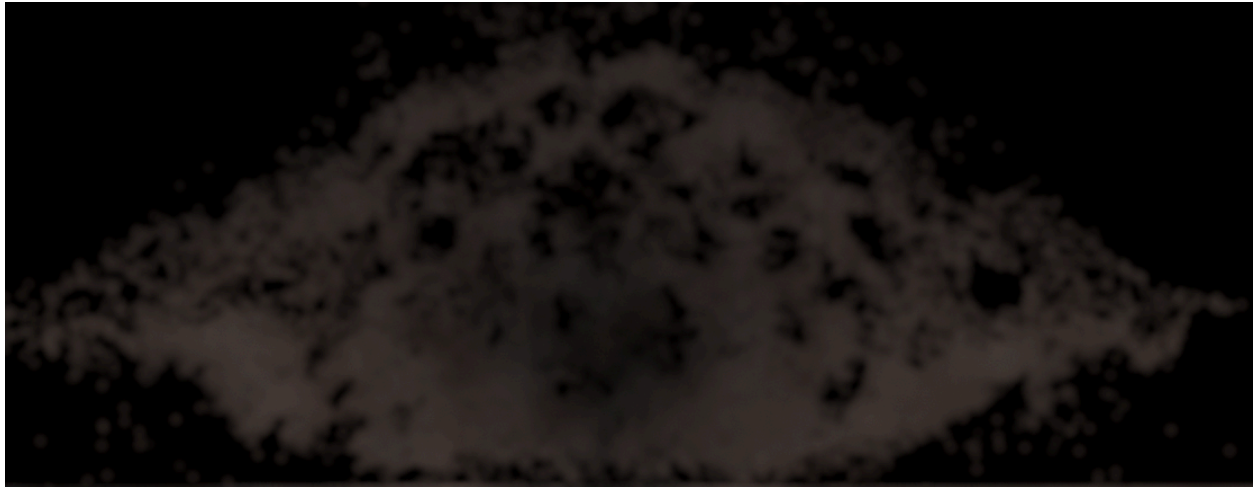


Figure 6: Cloud of smoke particles

Physical Properties	Visual Properties
• Light with low density and gas-like properties • Slowly disappears over time as temperature decreases • Generated by the combustion of specific particles (such as crude oil)	• Dark gray • Uses velocity shader to change tone when moved quickly

5. Cold Fire

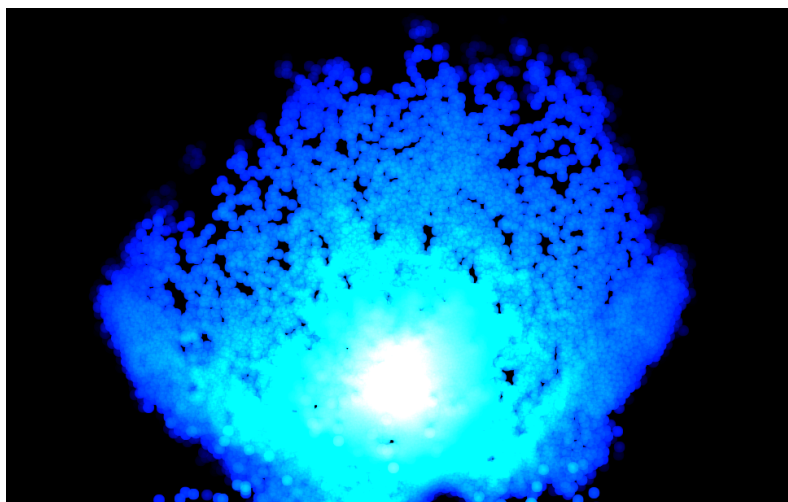


Figure 7: Cold fire particles

Physical Properties	Visual Properties
• Imaginary substance • Incredibly low base temperature • Light with low density and gas-like properties • Slowly disappears over time as temperature increases	• White-blue to dark blue, depending on temperature • Fades to black as it warms to prevent sudden disappearing effect • Glows

Level 2 Improvements

The graphics in level 2 have been updated to add more polish compared to before.

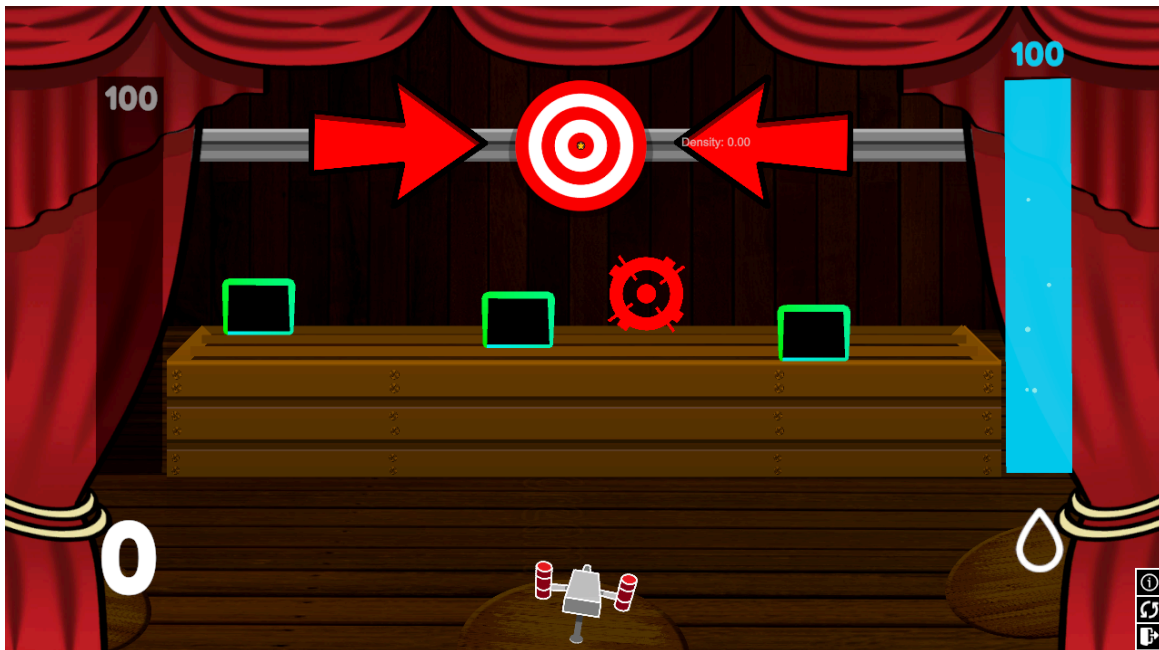


Figure 8: Level 2 with updated graphics

Level 3 Initial Implementation

Level 3 is currently under development, a development version of the level can be seen in the below figure.

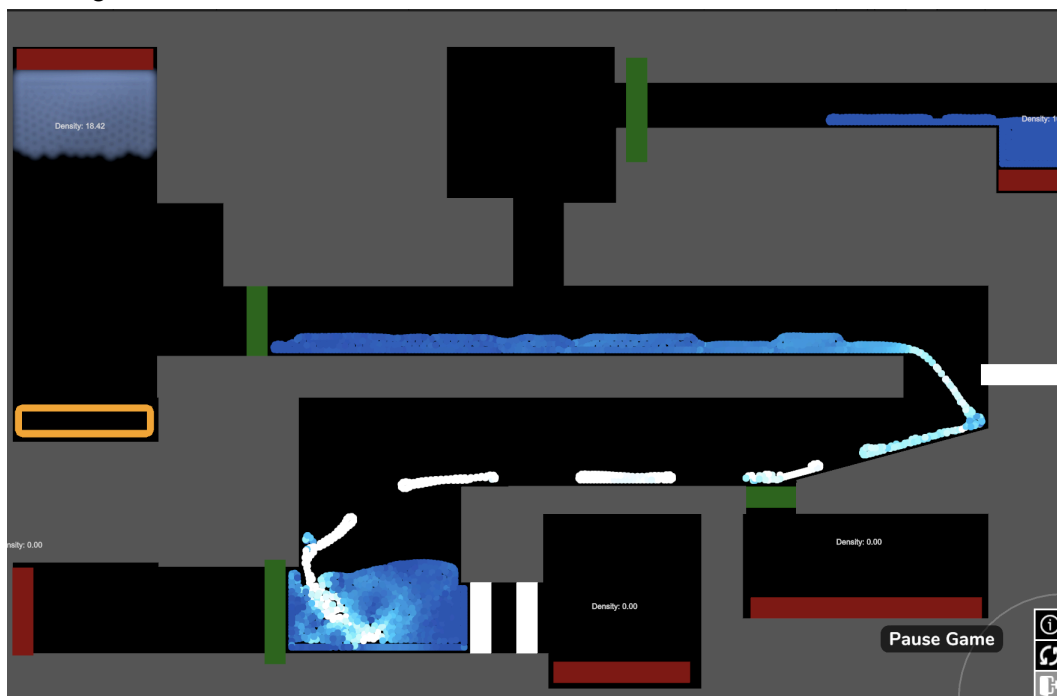


Figure 9: Preliminary Design of Level 3

Level 3 is a “trap door” type level in which the player interacts with various doors/gates to direct fluids towards detectors, which then unlock or otherwise trigger other events in the scene. Level 3 will be designed to showcase multi-fluid simulations and allow the player to interact with different types of fluids to complete the level. The level will also showcase temperature state changes, such as boiling water to create steam.

Level 4 Planning Stage:

By utilizing our temperature simulation we plan to have a level where the goal is to mix different temperatures of lava until the player gets the correct temperature.

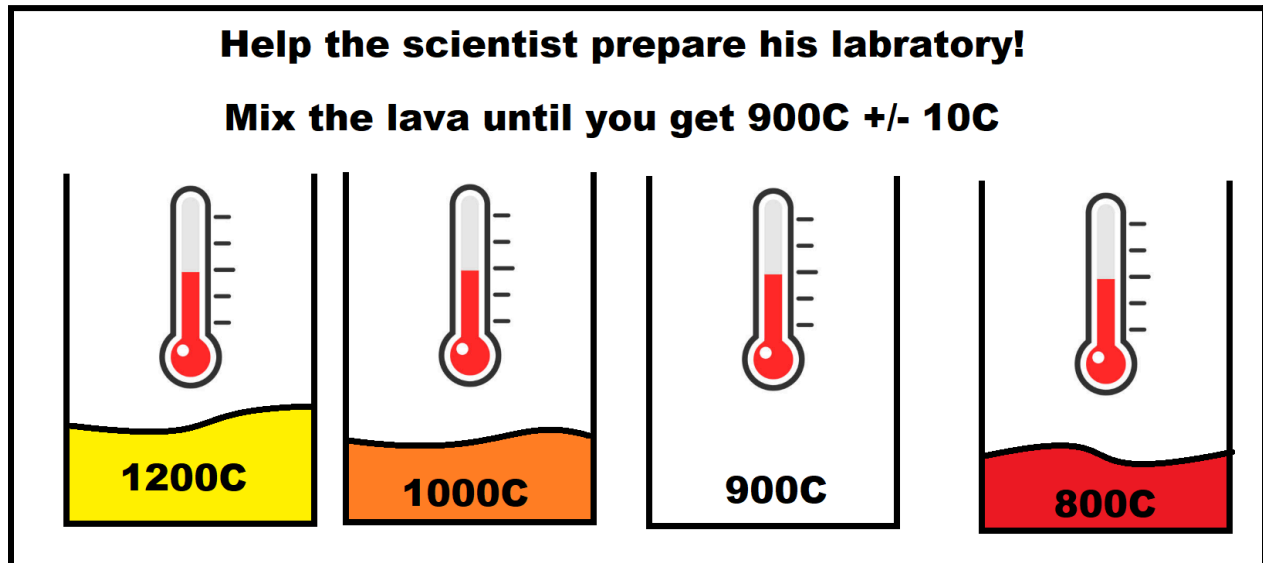


Figure 10: Concept Design for Level 4

Level 5:

We plan to have a level where two energy beams clash in the middle and the player has to add some interaction to have one beam 'overcome and defeat' the other.

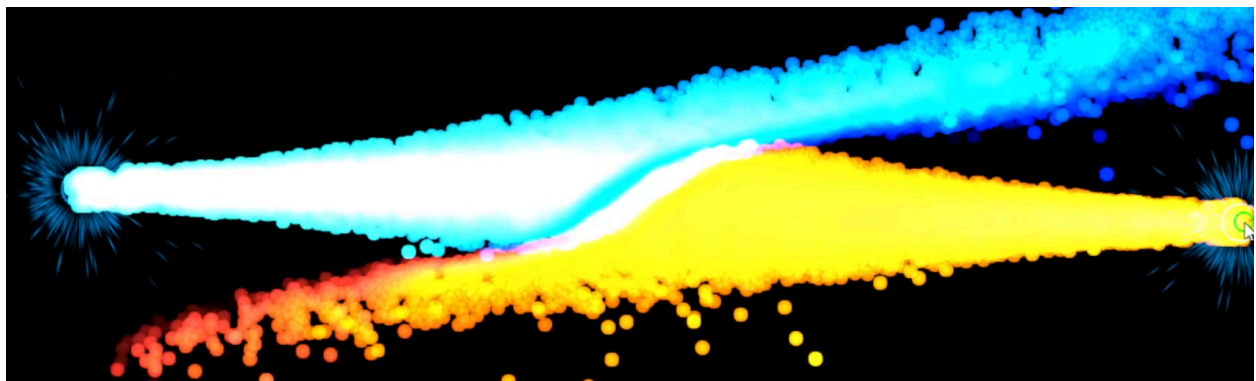


Figure 11: Concept Design of Level 5

Problems Encountered:

Porting to WebGPU

While we write our compute shaders in HLSL which is designed for DirectX, Unity helps us with cross compiling the shaders into other languages including into platform-specific languages like GLSL (OpenGL), Metal (iOS and macOS), or SPIR-V (Vulkan) [2]. This enables us to target different platforms without major rewrites of our codebase. This however can also introduce platform specific bugs and issues that we need to diagnose.

For instance, to support web platforms, Unity translates the shaders to WGSL which is a part of the new standard of WebGPU which is the successor of the older more well known WebGL standard [3]. It is lower level, supports more general purpose GPU computations outside traditional vertex and fragment shaders, supports more performance and it is stateless which reduces source of errors. It is designed to be completely asynchronous. An example of this is that it does not allow for synchronous fetch data commands as it can flush the pipeline and cause bubbles, a type of pipeline stall. So all readback commands must be made asynchronously. Another crucial feature of WebGPU over WebGL is that it supports compute shaders which our application needs for the physics and collision detection kernels. Compute Shaders are general purpose programs which run on the GPU to perform computational tasks. However, one drawback is that it is a new and emerging standard which is not widely supported on all web browsers yet. The current browser with the most support is Google Chrome, other browsers like Firefox and Safari have limited experimental support which you need to enable a feature flag for in the settings. As a result the embedded web player has lowered performance and stability on Safari compared to Chrome. This compounds with the fact that the embedded web player in Chrome already has less performance and more bugs compared with the standalone desktop applications which we can build for Windows, Android, MacOS, and Linux. The reason that we target this platform despite the issues is for greater accessibility as it allows the game to be played on the browser from devices without requiring any downloads.

Viscosity Clumping Issue:

Creating different fluid types using the combination of fluid properties is a balance between the repulsive and attractive forces. This can cause very situational issues that degrade the performance of the simulation. For instance, one of the most computationally heavy calculations that SPH, a purely lagrangian or particle based fluid simulation, has to do is the nearest neighbour search on each simulation step. This search causes performance to drop when particles are too densely spaced for long periods of time. We had an issue where when a particle such as steam was forced into a small space, the particle viscosity and pressure properties which work well normally would cause the particles to overlap and become unable to be separated. We implemented a checking stage that detects particles which are in unusually dense areas and disables the viscosity value for a single simulation step. This solution causes those clumped particles to rapidly disperse, restoring performance to the simulation.

High Pressure Instability:

One issue that caused instability and ultimately performance issues was the interaction between particles which had high pressure forces such as lava or honey. High pressure forces are desirable because they can cause these types of fluid to act very sticky and thick while preserving their overall volume from setting a high target density value. A problem this posed is that the desired effect was only achieved within large bodies of the fluids. This was because when a lot of the particles were accumulated together, each particle would be surrounded by others and the high pressure values would cancel each other out leading to a stable outcome where the entire volume of fluid would resist being stretched apart. An undesired effect was that when two singular honey or lava particles came into contact with each other, their repulsive forces would have nothing cancelling or tempering them which would cause them to 'bounce' off each other. They would fly around the simulation space continuously and gain momentum each time they met, indefinitely. To fix this we implemented a check within the pressure kernel which for arbitrarily high pressure values, would count the number of nearest neighbour particles and only apply the pressure if there were a moderate amount.

GPU Memory Management:

For CPU code which we write in C#, there is a garbage collector. That means if we create a new array or variable and point an existing variable to that array, losing the reference of the old array stored at that variable in the process, that the garbage collector will detect that the old array is unreachable and clear that memory for us. Unity even provides different settings for the garbage collector to work incrementally instead of clearing large amounts of memory in a single frame which can lead to visible hiccups and slowdowns which are undesirable.

In contrast, GPU memory doesn't have a garbage collector. For every new array or structured buffer we define and initialize we free the memory once that memory space is no longer needed. This comes up in three different scenarios, when a level is unloaded like when the player completes or quits a level and goes into the main menu and the reverse, when a new obstacle needs to be spawned into the scene spontaneously, and during development when we want the simulation to read changes we make to the particle properties immediately for testing purposes.

The simplest scenario is when a level is unloaded, unity provides a `OnDestroy()` callback which gets activated when we unload a scene. We just have to keep track of the buffers and textures that each script utilizes and free all that memory in the `OnDestroy()` callback.

The second simplest scenario is when the buffers must be updated each frame for testing purposes. During testing and development of a new fluid type we will rapidly adjust the particle properties such as its density and viscosity and we want those changes to apply to the simulation immediately so we can observe them. The issue is that at the start of a scene the buffer containing all the particle properties is created and sent to the gpu in addition an atlas texture containing all the particle visual properties is sent to the gpu. In order to update these buffers and textures we must update the old ones which involves destroying and freeing the old textures. This adds overhead which means that we disable this feature in actual builds since the

number of fluids and their properties will be predetermined by us the developers beforehand and will not change during the process of playing the game unless we choose to do so as part of the level design.

The final scenario where the buffers need to be destroyed and recreated is when obstacles are spawned in. We only initialize the buffers that contain obstacle struct information to be as large as needed to contain structs for each obstacle in the initial scene. That means if a new obstacle must be spawned in, a new buffer with a larger length must be defined and initiated with the information on each obstacle. To do this we use a callback that scans each object in the scene and uses the Unity tagging system which involves a simple integer flag check that sorts the obstacles into the appropriate array of structs according to its type i.e. box, circle, drain, source, or thermal box. Then each array of structs is used to initialize a structured buffer which is sent to the GPU. After that, each frame, that buffer needs to be updated with the current location and rotations and scales of each obstacle.

While there may be a few dozen obstacles per level there will be tens of thousands of particles per level. This means destroying and recreating the buffer that contains particles would induce an unacceptable amount of overhead for every interaction where the number of particles was changed. Thus we use an entirely different method of managing particle count that involves deactivation and activation rather than deletion and creation. This method is detailed in previous papers, to summarize it involves setting the particle type to 0 which indicates that it is disabled and then skipping over unnecessary calculations in each kernel and not displaying the disabled particles in the visual shader.

Next Steps:

Powders

We wish to create another type of particle which we will use to simulate powdery or aggregate or granular matter. This will be useful for simulating snow when water falls below a certain temperature and for simulating gravel which lava can transition to when falling below a certain threshold temperature. This will involve reworking the data structures a bit and adding the corresponding friction kernels to simulate cohesion forces.

More Fluid Types

Some ideas for other particles involve flammable natural gas, nuclear radioactive particles and high energy particles like neutrons, or electrons.

More Level Implementations

Future level implementations include a drink mixing game, a fire fighter level, a chemistry level, and pipes level. Each of these levels is still in early planning stages with no preliminary implementation, but the summary of each will be described in this section.

The drink mixing will involve the creation of various drinks using different quantities of fluids placed within box-collider “cups”, with the goal being to create several different drinks in

quick succession. Scoring will be based on accuracy with respect to the targeted drink, as well as time taken to create the drink.

The chemistry level will involve a chemistry set where the player has a few beakers and a bunsen burner and is tasked with boiling and distilling dirty water to achieve a high enough purity. Scoring will involve

The fire fighter level will involve having a small character moving around the scene with a water hose, extinguishing fires. Scoring will likely be based on time as well as quantity of water used.

Conclusion:

This milestone demonstrates substantial progress towards a robust and versatile fluid simulation. We have achieved major performance gains, expanded the range of simulatable phenomena, improved user interaction, and laid a solid foundation for engaging gameplay. While challenges in cross-platform compatibility and simulation stability have been addressed, ongoing efforts will focus on adding new particle types (powders and additional fluids), finalizing levels 3, 4, and 5, and refining existing systems. The project is well-positioned to deliver a compelling and interactive fluid simulation experience. The core physics engine is performant and our remaining goals will focus on content using that engine.

References

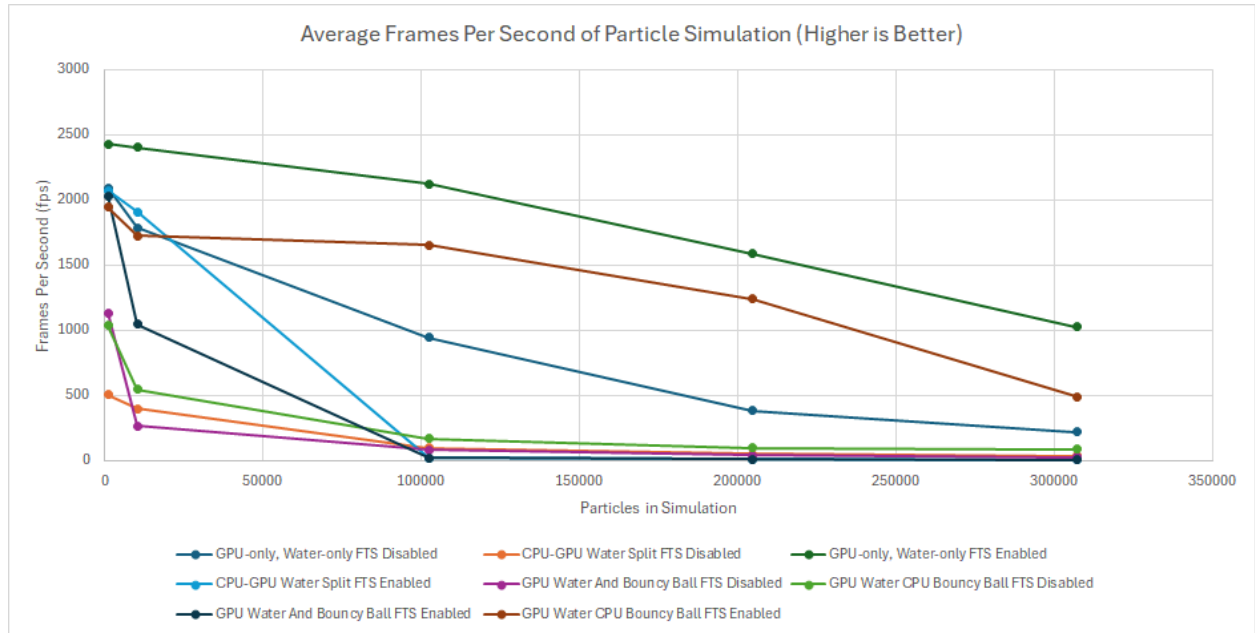
[1] U. Technologies, “AsyncGPUReadback.request,” Unity, <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Rendering.AsyncGPUReadback.Request.html> (accessed Mar. 13, 2025).

[2] U. Technologies, “Shader compilation,” Unity, <https://docs.unity3d.com/Manual/shader-compilation.html> (accessed Mar. 13, 2025).

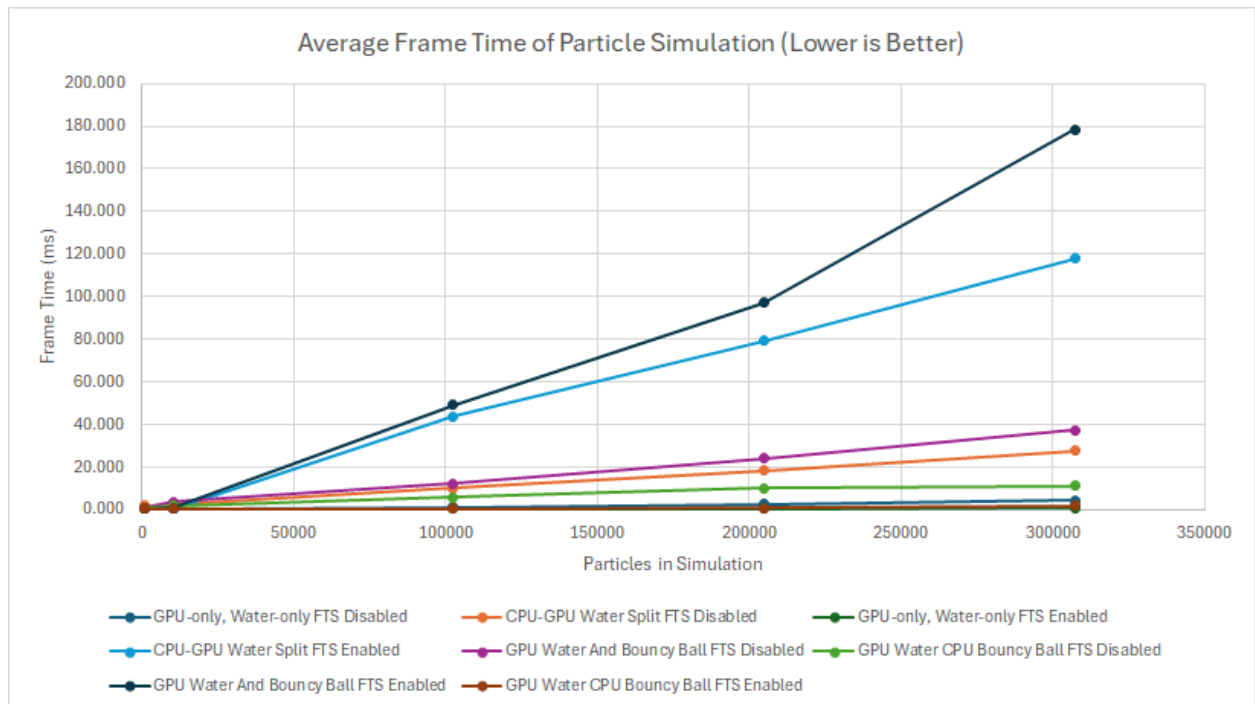
[3] F. Beaufort, “From WebGL to webgpu,” Chrome for Developers, <https://developer.chrome.com/blog/from-webgl-to-webgpu> (accessed Mar. 13, 2025).

Appendix:

Graph A-1: Performance Benchmarking Results - Average Frames Per Second (fps)



Graph A-2: Performance Benchmarking Results - Average Frame Time (ms)



Graph A-3: Performance Benchmarking Results - Average Latency Cost Per Particle (us)

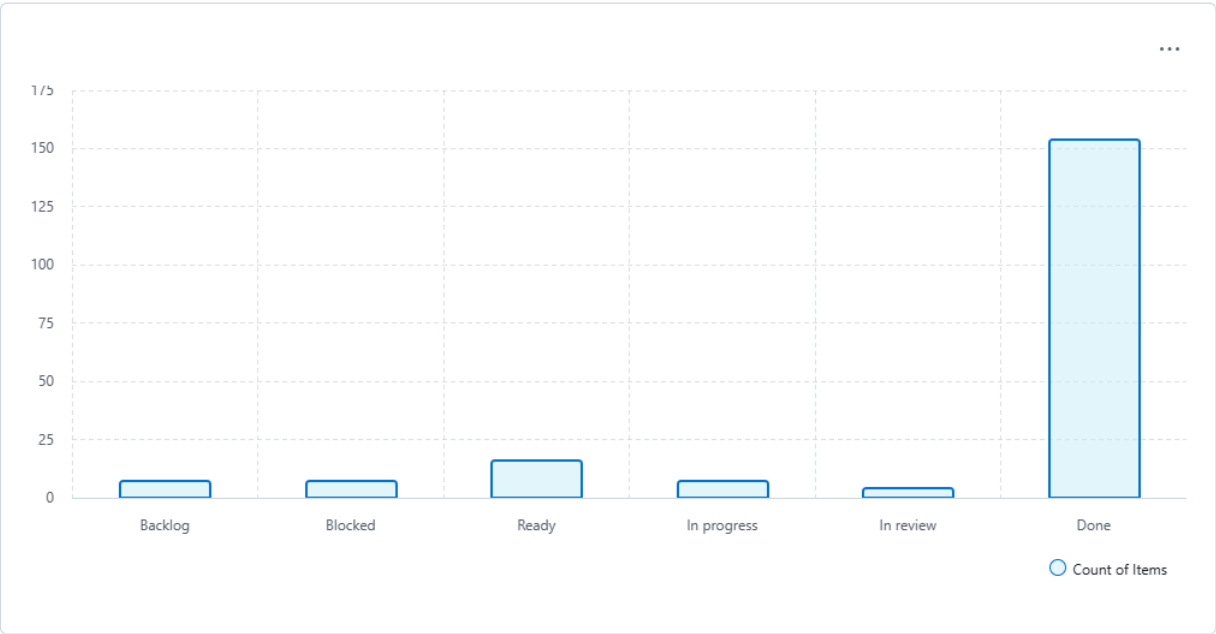
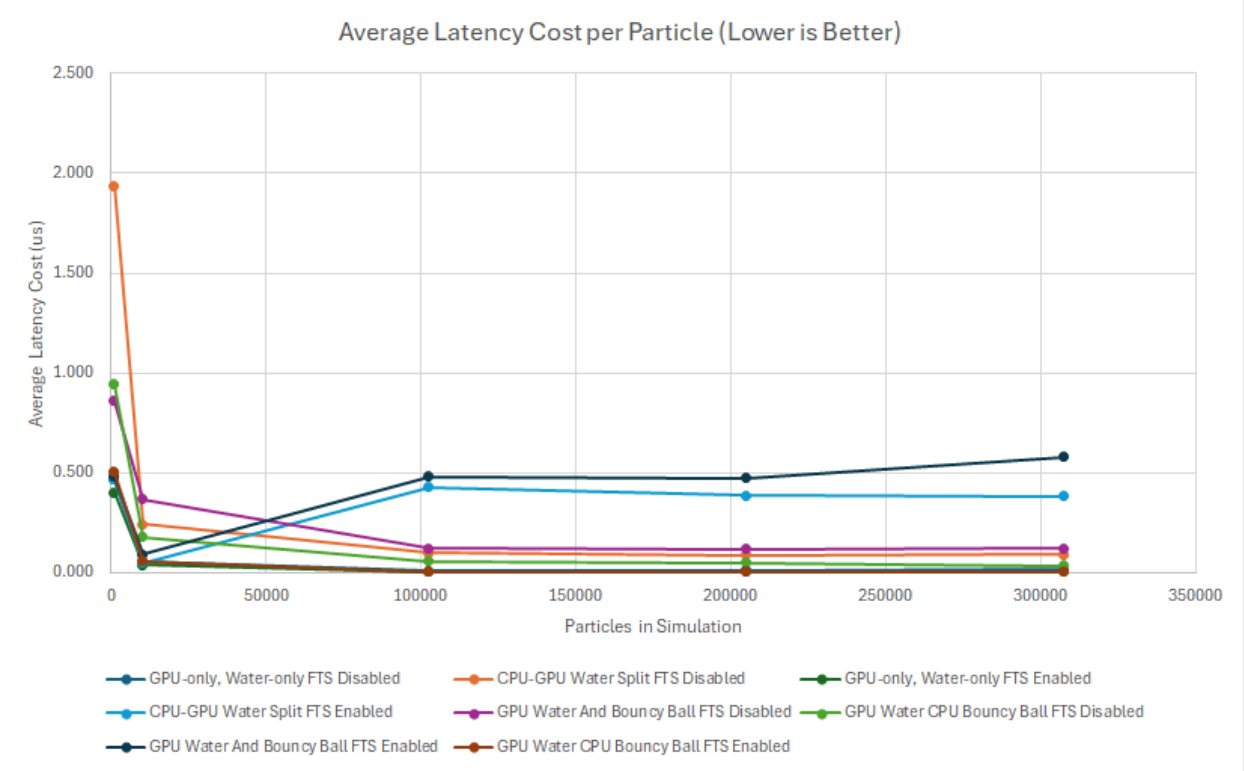


Figure 12: The current status of all tasks on our github repository.
(More Tasks will be created as is necessary)

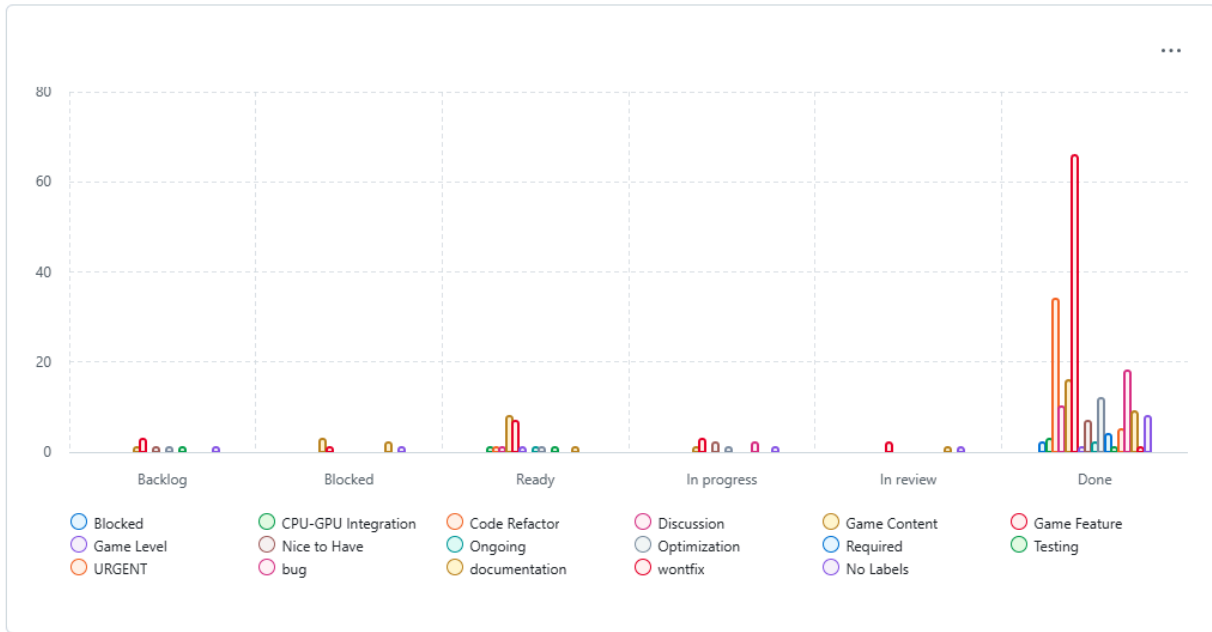


Figure 13: This is the breakdown of the current status of all tasks based on their type. Each bar from left to right corresponds to the legend labels from left to right.

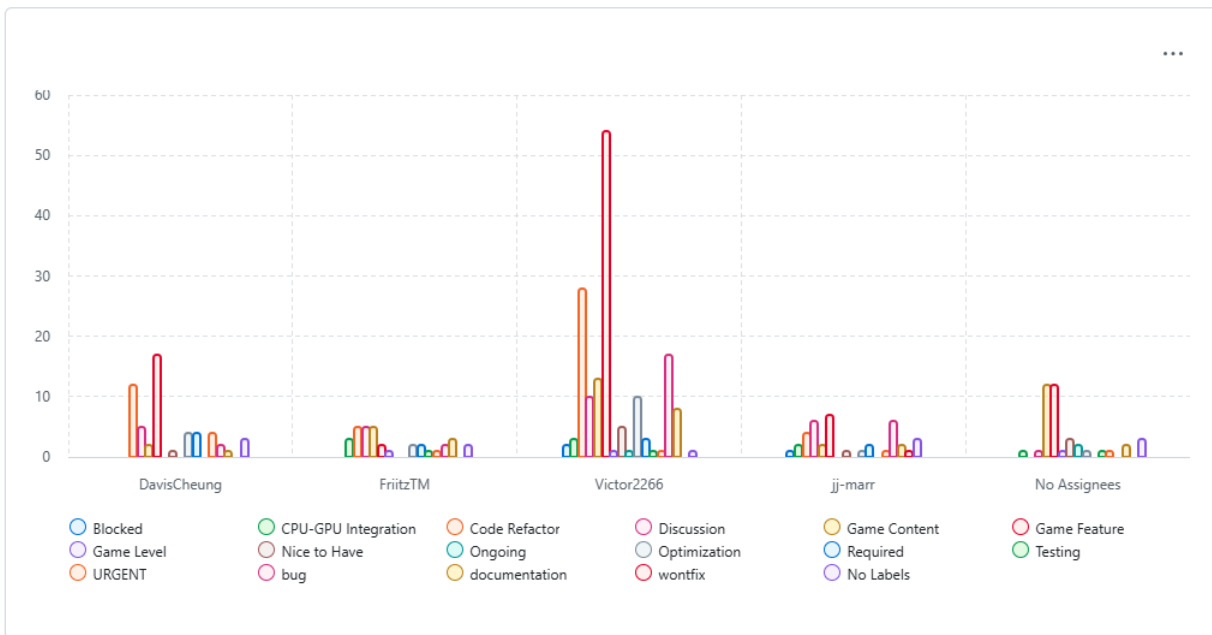


Figure 14: This is a breakdown of the types of tasks each member has worked on grouped by task type.

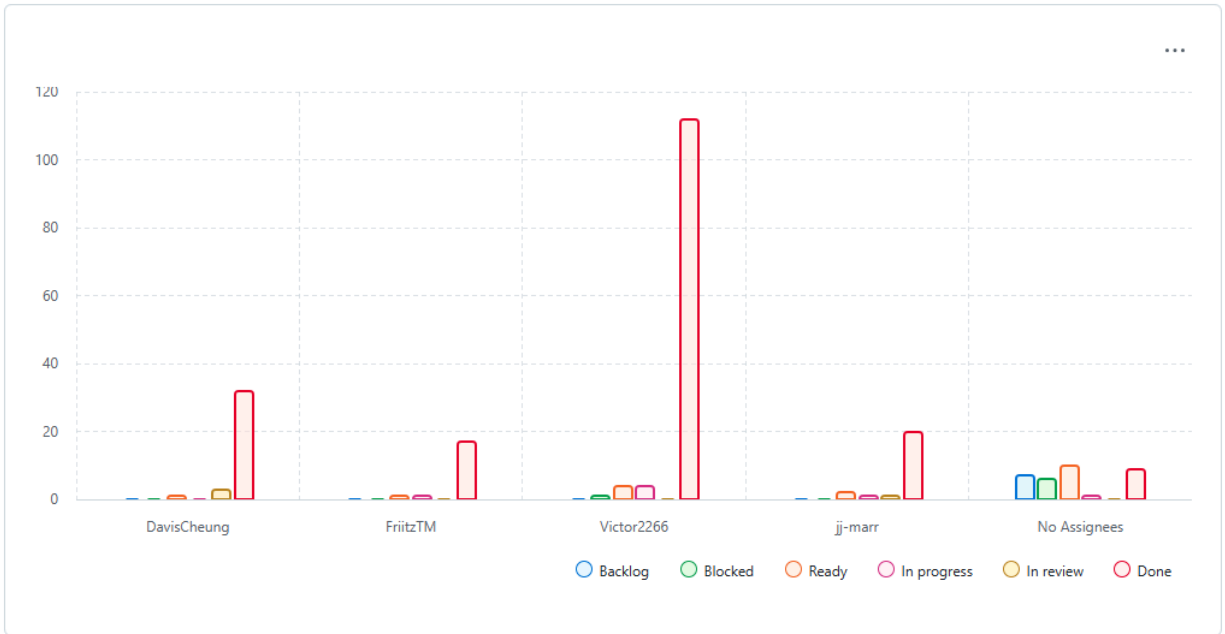


Figure 15: This is a breakdown of the status of tasks assigned to each member.

Contributors

Period: All

Contributions: Commits

Contributions per week to main, excluding merge commits

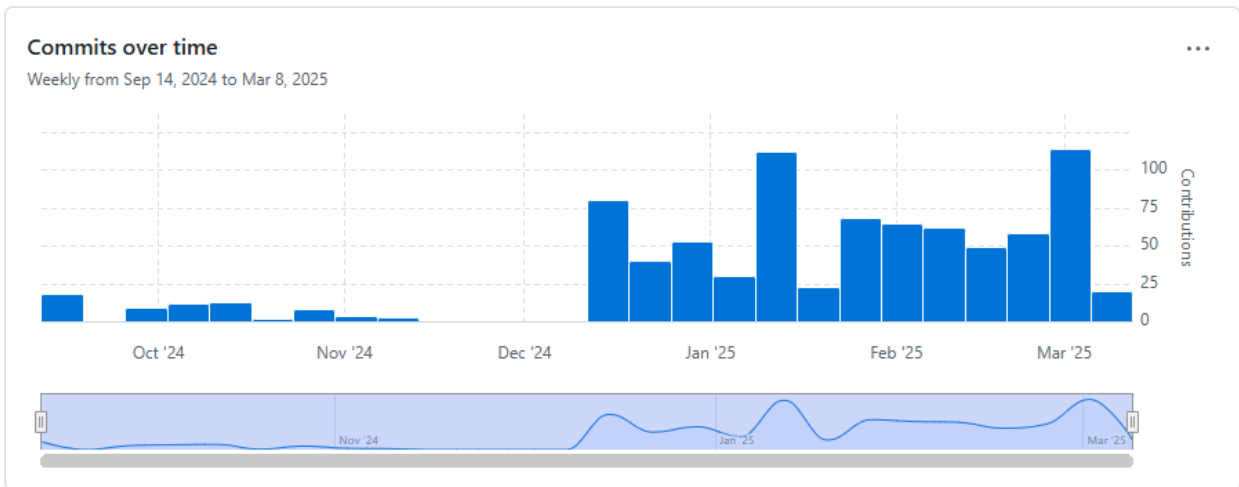


Figure 16: This is the commit history of the project.

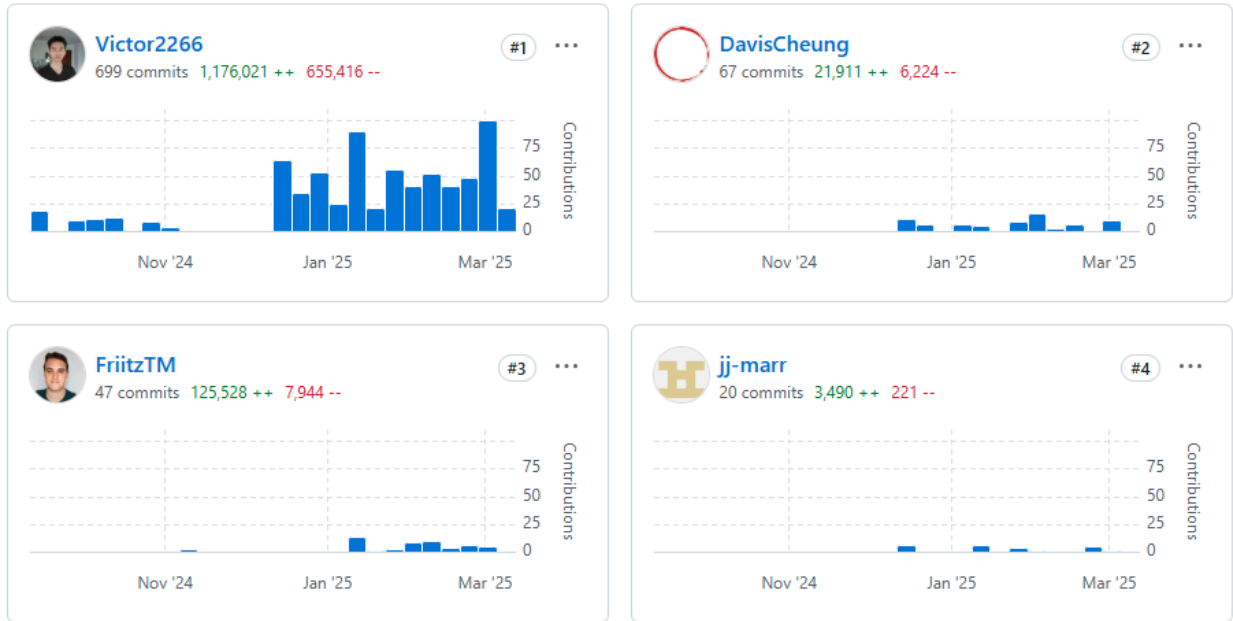


Figure 17: This is a breakdown of the commit history of each member

Burn up

[Configure](#)

The Burn up chart shows the progress of your project items over time, showing how much work has been completed and how much is left to do. Use this chart to view progress, spot trends, and identify bottlenecks to help move the project forward.

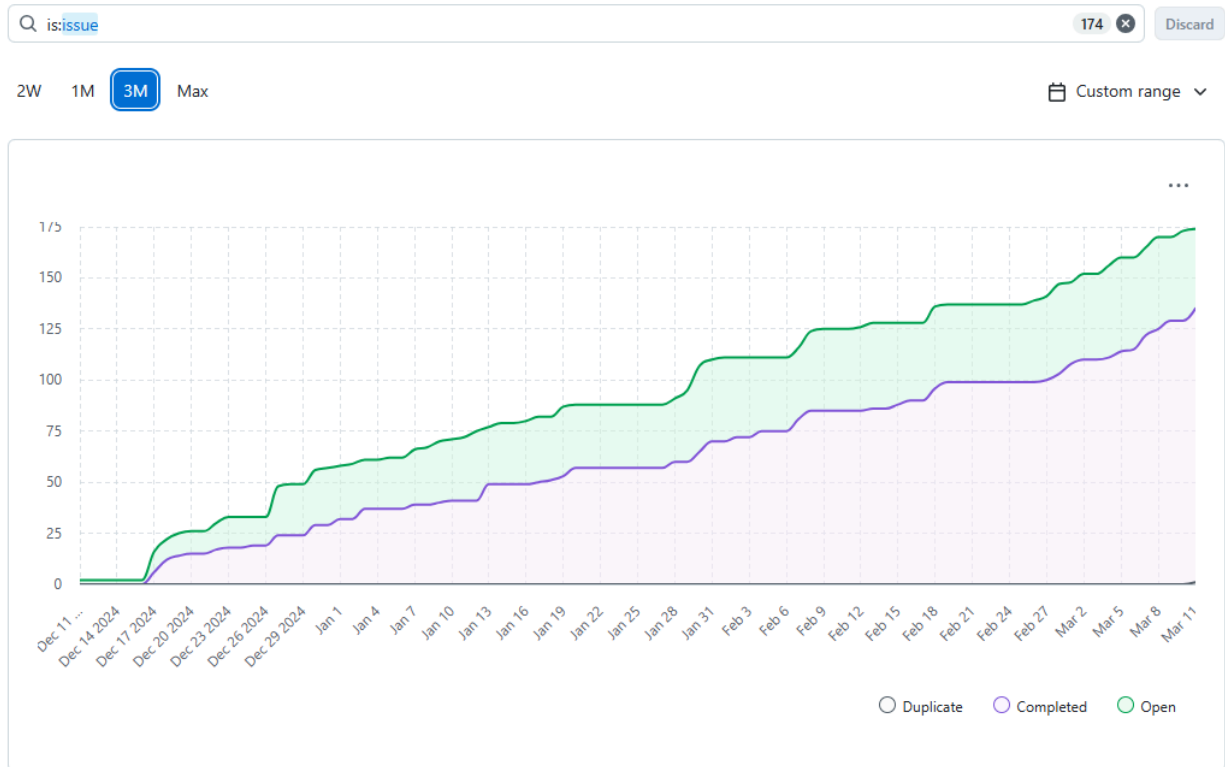


Figure 18: The Burn up chart shows the progress of the project items over time, showing how much work has been completed and how much is left to do. This showcases how we continually open and close new issues with an average of 40 open issues at any given time.